

Part 5 · The same ground, read from the source

Part 3 and Part 4 are what a person sees. This part is what the code says, mapped file by file by five independent readers working from the repository at `HEAD`. Every claim carries a `file:line`. Where a reader could not settle a question from code alone it is listed as an open question at the end of its section rather than guessed at.

Read this part when you want to know *why* a screen behaves as it does, or what a branch the device walk did not take would have shown.

5.1 · In the app — first launch to a usable account

The app from first launch to a usable account

Repo root `/Users/matan/Projects/soccer`. All paths below are absolute-from-repo-root. App version in `app.json` at the time of reading: `"version": "1.1.10"` (`/Users/matan/Projects/soccer/app.json`, `expo.version`).

Every Hebrew string is quoted verbatim from source with its `file:line`. Strings that live in `src/i18n/he.ts` are given as `key: 'value'` at their `he.ts` line, plus the line of the screen that renders them.

0. What runs before any pixel of ours is drawn

These fire at **module load of** `App.tsx`, i.e. before React mounts, before the user has seen or agreed to anything.

Order	What	File:line	Notes
1	<code>void joryio.init();</code>	<code>App.tsx:17</code>	Joryio analytics/messaging SDK starts before the first screen, before sign-in, before any consent UI . Implementation: <code>src/services/joryio.ts:103–118</code> (<code>initJoryio</code> → <code>Joryio.initialize(KEY, API_HOST, {...})</code>). SDK keys are hardcoded literals as fallback at <code>src/services/joryio.ts:39–49</code> .
2	<code>ExpoSplash.preventAutoHideAsync()</code>	<code>App.tsx:19–21</code>	Holds the OS splash up until our React splash paints.
3	<code>LogBox.ignoreLogs([...])</code>	<code>App.tsx:29–39</code>	Dev-only noise suppression.
4	<code>Notifications.setNotificationHandler({...})</code>	<code>App.tsx:67–137</code>	Foreground push behaviour. Does not request permission.
5	<code>Notifications.setNotificationCategoryAsync('NEW_GAME_RSVP', [...])</code>	<code>App.tsx:164–175</code>	Registers the action button <code>buttonTitle: 'אני חגיע'</code> (<code>App.tsx:167</code>).

Order	What	File:line	Notes
6	<code>Notifications.setNotificationCategoryAsync('SPOT_OFFER', [...])</code>	<code>App.tsx:180–193</code>	Action buttons <code>buttonTitle: 'מאשר' (App.tsx:185)</code> and <code>buttonTitle: 'ויתור' (App.tsx:190)</code> .
7	<code>I18nManager.allowRTL(true) / forceRTL(true)</code>	<code>App.tsx:253–258</code>	RTL forced on first launch. Comment at <code>App.tsx:256–257</code> notes a forced RTL switch normally needs a JS reload.
8	Global Text / <code>TextInput defaultProps → textAlign:'right', writingDirection:'rtl'</code>	<code>App.tsx:274–295</code>	Every string in the app is right-aligned by default.
9	<code>installGlobalErrorHandlers()</code>	<code>App.tsx:305</code>	Crash + unhandled-rejection catch-all.

Firebase App Check initialises lazily the first time `getFirebase()` runs — `src/firebase/config.ts:156–163` (`const { initAppCheck } = require('./appCheck');` `void initAppCheck(_app);`).

Firebase Analytics is the native `@react-native-firebase/analytics` SDK (`src/services/analyticsService.ts:13–17`); there is no in-app analytics opt-in before it collects.

There is no App Tracking Transparency prompt on iOS. Verified by grep across `src`, `App.tsx`, `app.json`, `plugins:no requestTrackingPermissionsAsync`, `no expo-tracking-transparency`, `no NSUserTrackingUsageDescription`.

There is no AdMob UMP / GDPR consent form. `adsService.initializeAds()` (`src/services/adsService.ts:327–394`) calls `sdk.initialize()` (`:379`) and creates the app-open ad with `requestNonPersonalizedAdsOnly: true` (`:383`) — no consent-info-update, no form.

1. Screen 1 — Native OS splash

- File / config:** `/Users/matan/Projects/soccer/app.json` → `expo.splash = {"image": "./splash-blank.png", "resizeMode": "contain", "backgroundColor": "#1E40AF"}.`
- Shown:** always, at process start, on every launch.
- Content:** the image file is literally named `splash-blank.png` — the visible result is a flat `#1E40AF` blue field.
- Strings:** none.
- Controls:** none. Not skippable, not backable.
- Dismissed by:** `ExpoSplash.hideAsync()` called from inside our React splash's first effect — `src/screens/SplashScreen.tsx:372–374`. The comment at `App.tsx:393–397` explains this is deliberate, so there is no black flash between native dismiss and React first paint.
- Android adaptive icon / Android splash background also `#1E40AF` (`app.json`, `expo.android.adaptiveIcon.backgroundColor`).

2. Screen 2 — Animated splash (SplashScreen)

- **File:** `/Users/matan/Projects/soccer/src/screens/SplashScreen.tsx`
- **Mounted from:** `App.tsx:1006-1011` — `{!splashDone ? <SplashScreen ready={userHydrated && (!currentUserId || groupHydrated)} onFinish={handleSplashFinish} /> : null}`. It renders **over** `RootNavigator`, which mounts and hydrates behind it (`App.tsx:989-1005`).
- **Shown:** always, every cold start.
- **Duration:** minimum hold `const MIN_HOLD_MS = 1400;` (`src/screens/SplashScreen.tsx:41`), then a `FADE_MS = 320` fade-out (`:42`). The fade only starts once `ready` is true (`:376-398`). For a signed-out fresh install, `ready = userHydrated` alone, because of the `!currentUserId` short-circuit at `App.tsx:1008` (documented at `App.tsx:996-1005` — waiting on `groupHydrated` would pin the splash forever for a signed-out user, since `hydrateGroup` only runs once a `currentUser` exists, `RootNavigator.tsx:201-211`).
- **Animation sequence** documented at `src/screens/SplashScreen.tsx:3-13`: pitch lines draw on (~480-700ms), ball kicked at the camera from `KICK_AT = 440` (`:105`), wordmark rises at ~1100ms.
- **Strings, verbatim:**
 - `Teamder` — `src/screens/SplashScreen.tsx:285` (hardcoded, `allowFontScaling={false}`)
 - `המשחק הבא שלך מתחיל כאן` [your next game starts here] — `src/screens/SplashScreen.tsx:288` (hardcoded, **not** in `he.ts`)
 - Three pulsing dots, no text — `:290-294`
- **Controls:** none. No tap advances or skips it.
- **On finish:** `handleSplashFinish` (`App.tsx:871-903`). Critically, **the app-open ad is skipped entirely when there is no signed-in user** — `if (useUserStore.getState().currentUser) { ... }` at `App.tsx:879`, with the rationale at `App.tsx:872-878`: "a fresh install (incl. the Play reviewer) goes straight to the app, never sitting on the splash behind an ad." Even for signed-in users the ad is raced against a 3500ms hard timeout (`App.tsx:884-892`).
- **Second appearance:** `RootNavigator` renders the same visual again via `SplashVisual` (`src/navigation/RootNavigator.tsx:274`, `:287-289`) when `!groupHydrated` after sign-in. So a brand-new user sees the ball splash a **second** time between tapping "המשך" on the profile screen and landing on the tabs.

3. The decider — RootNavigator

File: `/Users/matan/Projects/soccer/src/navigation/RootNavigator.tsx`. This is the single branch point. Exact order of the gates (`:251-279`):

```
251 if (!userHydrated)                → <Splash />
253 if (!onboardingDone)              → <OnboardingScreen />
255 if (!currentUser)                 → <AuthStack initialRoute="SignIn" />
260 const isGuest = currentUser.isGuest === true
267 if (!isGuest && currentUser && !hasCompletedOnboarding) → <PostSignInOnboardingScreen />
271 if (!isGuest && !profileComplete) → <AuthStack initialRoute="ProfileSetup" />
274 if (!groupHydrated)               → <Splash />
279 return <MainTabs />
```

Gate definitions: - `onboardingDone` — hydrated from AsyncStorage key `ONBOARDING_DONE`:
'footy.onboarding.done' (`src/services/storage.ts:7`, read at `:84–86`, written at `:88–90`). Loaded in
`useUserStore.hydrate` (`src/store/userStore.ts:109–122`). **Sign-out does NOT clear it**
(`src/store/userStore.ts:164–187` wipes group/game/chat stores only), so the intro slides are shown **once per install**, never again. - `profileComplete` — `isProfileComplete: () => !!u && u.name.trim().length > 0`
(`src/store/userStore.ts:243–246`). - `hasCompletedOnboarding` — `!!u && u.onboardingCompleted === true` (`src/store/userStore.ts:248–251`).

Side-effects mounted here, with their firing conditions: - `hydrateUser()` + `void initRemoteConfig()` +
`adsService.initializeAds()` on mount — `RootNavigator.tsx:170–181` . - App-open ad once on MainTabs,
only when `membership === 'member' && gameStatus !== 'locked'` — `:186–193` . - `hydrateGroup /`
`subscribeGroups / hydratePlayers` once `currentUser` exists — `:201–218` . - Live `/users/{uid}` listener —
`:226–229` . - **Push permission request** — `:235–248` . See §7. - Pending-invite deep-link consumer — `:63–166` .
Error toast string: `toast.error('הקישור לא תקין או שהפריט כבר לא קיים')` — `RootNavigator.tsx:124`
(hardcoded, not in `he.ts`).

4. Screen 3 — Pre-sign-in onboarding (`OnboardingScreen`)

- **File:** `/Users/matan/Projects/soccer/src/screens/onboarding/OnboardingScreen.tsx`
- **Shown when:** `!onboardingDone` (`RootNavigator.tsx:253`) — i.e. exactly once per install, before any auth.
- **Skipped when:** the AsyncStorage flag is already `true` . A signed-out returning user on the same install goes **straight to SignIn**, never sees the slides again.
- **Layout:** full-screen `LinearGradient` with `const GRADIENT = ['#0F172A', '#1E3A8A', '#1E40AF']` (`:55`).
Comment at `:1–6` records that this is a trim from an earlier **4-slide green** version to "the 3 highest-signal pitches".
- **Carousel:** horizontal paging `FlatList` with `inverted` (`:122–144`, `inverted` at `:129`) — inverted because of forced RTL, so the user swipes right-to-left.
- **Pagination:** three animated dots, width interpolating 8→24dp (`:146–150`, `Dot` at `:192–216`).

SLIDE CONTENTS (`SLIDES` ARRAY AT `:43–47`)

Slide 1 — preview `PreviewMatches` (`:44`) - Image: `assets/images/onboarding/games.png` —
`src/screens/onboarding/OnboardingPreviews.tsx:37` - Title, rendered at `OnboardingScreen.tsx:139` :
`onb1Title: 'שחקו עם אנשים בקרבת מקום'` — `src/i18n/he.ts:1781` - Body, rendered at `:140` : `onb1Body:`
'גלו מחזורי כדורגל פתוחים באזור שלכם והצטרפו בלחיצה – או פגשו שחקנים חדשים לידכם' —
`src/i18n/he.ts:1782`

Slide 2 — preview `PreviewClub` (`:45`) - Image: `assets/images/onboarding/club.png` —
`OnboardingPreviews.tsx:41` - `onb2Title: 'מועדון קבוע, מחזור אוטומטי'` — `src/i18n/he.ts:1783` -
`onb2Body: 'בנו את הסגל הקבוע שלכם – והמחזור השבועי נפתח לבד עם הזמנה לכולם'` —
`src/i18n/he.ts:1784`

Slide 3 — preview `PreviewLive` (`:46`) - Image: `assets/images/onboarding/live.png` —
`OnboardingPreviews.tsx:45` - `onb3Title: 'הכל זורם מעצמו'` — `src/i18n/he.ts:1785` - `onb3Body: 'מישהו`
'ביטל? המקום מתמלא אוטומטית מרשימת ההמתנה עם תזכורות חכמות שדואגות שכולם יגיעו

`src/i18n/he.ts:1786`

The three previews are **real cropped screenshots of the live app** shown inside a drawn phone bezel — `OnboardingPreviews.tsx:1-5` ("REAL captures of the live app ... Source PNGs live in assets/images/onboarding/ (600×1253, status bar / ad / gesture bar stripped"), frame at `:28-34`, `FRAME_W = Math.min(236, SCREEN_W * 0.62)` (`:21`).

CONTROLS

Control	Label (verbatim)	he.ts line	Rendered at	Behaviour
Skip pill, top-leading	<code>onbSkip:</code> 'דלג'	<code>src/i18n/he.ts:1775</code>	Onboarding Screen.tsx: 117	<code>handleSkip (:100-103) → logEvent(AnalyticsEvent.OnboardingSkipped, { slide: index + 1, total: SLIDES.length }) then completeOnboarding() . Rendered only when !isLast (:115)</code> — there is no skip on slide 3.
Primary CTA, slides 1-2	<code>onbNext:</code> 'הבא'	<code>src/i18n/he.ts:1776</code>	Onboarding Screen.tsx: 180	<code>advance() (:73-77) → scrollToIndex(index+1) .</code>
Primary CTA, slide 3	<code>onbCtaStart:</code> 'המשך'	<code>src/i18n/he.ts:1780</code>	Onboarding Screen.tsx: 167	<code>handleStart (:84-94) → logEvent(AnalyticsEvent.OnboardingCompleted, { via: 'cta', slide: index + 1 }) → await completeOnboarding() . Disabled while busy .</code>

The slides are **purely informational** — no sign-in button lives here. Comment at `:79-83` : "This avoids the old duplication where both the last slide AND the sign-in screen carried login buttons."

DEAD / UNUSED STRINGS STILL IN HE.TS

These exist but nothing on this screen renders them: `onbStart:` 'בוא נתחיל' (`:1777`), `onbCtaSignIn:` 'התחבר עם Google' (`:1778`), `onbCtaSignInApple:` 'התחבר עם Apple' (`:1779`), `onb4Title:` 'בוא נתחיל' (`:1789`), `onb4Body:` 'התחבר ותתחיל לארגן מחזוריים' (`:1790`) — explicitly flagged as legacy at `he.ts:1787-1788` .

REQUIRED VS SKIPPABLE, BACKING OUT

- Required: **no**. One tap on 'דלג' clears the whole thing.
- Android hardware back on this screen: no stack to pop and we are not on a tab, so `useAndroidBack` (`App.tsx:352`, implementation `src/navigation/useAndroidBack.ts`) falls to rule 3 — first press arms + shows a toast `backAgainToExit:` 'לחץ שוב כדי לצאת' (`src/i18n/he.ts:51`), second press within `CONFIRM_WINDOW_MS = 2000` (`useAndroidBack.ts:47`) leaves the app. iOS: no hardware back (`useAndroidBack.ts:20-21`, `:69`).
- Once completed or skipped, **the slides are unreachable** — there is no "show intro again".

PERMISSIONS / SDK ON THIS SCREEN

None fire here. Joryio already started at module load (§0). No push prompt yet (guarded on `currentUser`, `RootNavigator.tsx:236`).

ERROR STATES

None. `handleStart` swallows failures (`:89-91` `catch { // no-op }`), so a failed `AsyncStorage` write silently leaves the user on the slides with the CTA re-enabled.

5. Screen 4 — Sign-in (SignInScreen)

- **File:** `/Users/matan/Projects/soccer/src/screens/auth/SignInScreen.tsx`
- **Shown when:** `onboardingDone && !currentUser` → `<AuthStack initialRoute="SignIn" />` (`RootNavigator.tsx:255`). Stack config: `src/navigation/AuthStack.tsx:19–29`, `screenOptions={{ headerShown: false }}` (`:23`).
- **Palette:** `const ACCENT = '#1E40AF'; const ACCENT_SOFT = '#DBEAFE';` (`:28–29`), deliberately not the legacy green `colors.primary` (`:24–27`).
- **Hero:** a 140dp circle (`:281–289`) containing `<Icons name="football-outline" size={72} color={ACCENT} />` (`:177`).

STRINGS, VERBATIM

Element	Value	he.ts	Rendered
Title	<code>signInTitle: 'בואו נתחיל'</code>	<code>:179</code> <code>3</code>	<code>SignInScreen.tsx</code> <code>:179</code>
Subtitle	<code>signInSubtitle: 'התחבר כדי להצטרף למועדון ולעקוב אחרי הסטטיסטיקות שלך.'</code>	<code>:179</code> <code>4</code>	<code>:180</code>
Google button	<code>signInGoogle: 'המשך עם Google'</code>	<code>:179</code> <code>5</code>	<code>:204</code>
Apple button	<code>signInApple: 'המשך עם Apple'</code>	<code>:179</code> <code>6</code>	<code>:229</code>
Email button	<code>signInEmail: 'המשך עם מייל'</code>	<code>:179</code> <code>7</code>	<code>:251</code>
Guest link	<code>signInGuest: 'המשך כאורח'</code>	<code>:179</code> <code>8</code>	<code>:269</code>
Footer	<code>signInPrivacy: 'באמצעות התחברות אתה מסכים לתנאי השימוש'</code>	<code>:179</code> <code>9</code>	<code>:272</code>

Note the mixed voice: the title `'בואו נתחיל'` is plural/formal, the subtitle `'התחבר...'` is masculine-singular imperative. Same tension appears between the slides (`'שחקו'`, `'גלו'`, `'בנו'` — plural) and everything after sign-in (`'בוא נכיר'` — masculine singular).

The privacy footer is a plain non-tappable `<Text>` (`:272`, style `privacy` at `:324`). It is **not** a link. Grep across `src/i18n/he.ts` for `privacy/terms` returns only this one string. A `public/privacy.html` exists on the hosting site but nothing in the in-app sign-up flow links to it.

CONTROLS AND BRANCHES

1. `'Google עם המשך'` — Pressable at `:188–207`. `handlePress` (`:44–93`): -
`logEvent(AnalyticsEvent.SignInAttempted, { method: 'google' })` (`:45`) -
`setBusyProvider('google')` → in-button `<ActivityIndicator />` (`:199–200`) - `signIn()` →
`userStore.signInWithGoogle` (`src/store/userStore.ts:130–134`) → `userService.signInWithGoogle`
(`src/services/userService.ts:167–212`) → `src/firebase/auth.ts:79–...` :
 - Android only: `GoogleSignin.hasPlayServices({ showPlayServicesUpdateDialog: true })`
(`auth.ts:93–95`) — can raise a **system Play Services update dialog**.

- `GoogleSignIn.signIn()` (`auth.ts:97`) — **system Google account chooser sheet**. One tap to pick an account.
 - `signInWithCredential` (`auth.ts:110`), plus `mirrorToNativeAuth` (`:112`) for the widget/watch.
 - New user → doc written at `userService.ts:181–194` with `name: fbUser.displayName ?? ''`, `email`, `avatarId: pickRandomAvatarId()`, `createdAt: Date.now()`, `onboardingCompleted: false` (`:187`). So a Google user arrives at the next screen **with their name pre-filled**.
 - Then `applyInviteAttributionIfFresh` + `applyAcquisitionIfFresh` (`:208–209`).
 - If the doc write fails, the user is **signed back out** (`:193–207`) to avoid an auth user with no `/users` doc.
 - `logEvent(AnalyticsEvent.SignInSuccess)` (`userStore.ts:133`).
 - Returning Google user → existing doc returned as-is (`userService.ts:176–180`), so `onboardingCompleted` stays `true` and the post-sign-in screen is skipped.
2. 'Apple עם המשך' — `Pressable` at `:212–233`, wrapped in `{Platform.OS === 'ios' && ...}` (`:212`). **Android users never see this button.** `handleApple` (`:97–139`). New-user doc at `userService.ts:228–239`, `name: fbUser.displayName ?? fullName ?? ''` (`:234`) — the comment at `:230–233` notes Apple only returns the name on the *first* authorization. `usesAppleSignIn: true` in `app.json` (`expo.ios`).
3. 'המשך עם מייל' — `Pressable` at `:236–252`. `logEvent(AnalyticsEvent.SignInAttempted, { method: 'email' })` then `nav.navigate('EmailAuth')` (`:238–239`). This is the **only** control here that is a navigation rather than an auth attempt. It carries no busy state (`:244–246` only dims when another provider is busy).
4. 'המשך כאורח' — underlined text link, `Pressable` at `:255–271`, style `guestText` with `textDecorationLine: 'underline'` (`:330–335`). `handleGuest` (`:143–155`) → `userStore.signInAsGuest` (`userStore.ts:142–146`) → `userService.signInAsGuest` (`src/services/userService.ts:61–67`) → `fbSignInAnonymously` (`src/firebase/auth.ts:407–412`). Builds a **runtime-only** user (`userService.ts:44–53`): `name: ''`, `avatarId: pickRandomAvatarId()`, `onboardingCompleted: true`, `isGuest: true`. **No /users doc is created.** Rationale at `userService.ts:56–60` and `SignInScreen.tsx:141–142`: App Store guideline 5.1.1(v). - Consequence at `RootNavigator.tsx:260–271`: a guest **skips both the post-sign-in profile screen and the profile-setup screen** and lands on `MainTabs` in one tap.

EVERY ERROR STATE ON THIS SCREEN

Error dialogs use `appAlert(he.error, ...)` where `error: 'שגיאה'` (`src/i18n/he.ts:188`). The message is resolved by `friendlySignInError` (`SignInScreen.tsx:158–171`):

Condition	Message	he.ts
cancelled	<code>signInCancelled: 'ההתחברות בוטלה'</code>	<code>:1262</code>
'OAuth client ID not configured'	<code>signInConfigMissing: 'עדיין לא מוגדרות Google הגדרות'</code>	<code>:1263</code>
browser blocked (<code>isBrowserBlocked</code>)	<code>signInBrowserBlocked: 'לא ניתן לפתוח את הדפדפן במכשיר הזה, נחסמה. בדרך כלל זה מגיע מהגבלת תוכן Google ולכן ההתחברות עם Apple בזמן מסך" או מפרופיל ניהול. אפשר להתחבר עם אימייל במקום.'</code>	<code>:1268–1269</code>

Condition	Message	he.ts
network / offline / auth/network- request-failed / unavailable	signInNetworkError: 'אין חיבור לאינטרנט'	:1270
anything else	signInFailed: 'ההתחברות נכשלה. נסה שוב.'	:1264
guest failure	signInFailed (direct, not via the mapper)	SignInScreen.tsx :151

Important behavioural detail: **a user-cancelled Google or Apple chooser shows no dialog at all** — `if (!cancelled) { showAlert(...) }` at `:87–89` and `:133–135`. It just returns to the sign-in screen with the spinner cleared.

Analytics on failure paths: `SignInCancelled` / `SignInFailed` (`:78–82` Google, `:126–130` Apple, `:150` guest). Transient errors are deliberately kept out of the error panel (`:63–77` Google, `:113–125` Apple).

REQUIRED VS SKIPPABLE, BACKING OUT

- Some auth choice is **required** to proceed past this screen — but "guest" satisfies it, so no account is strictly required to reach the app.
- Android back on SignIn: nothing to pop, not on a tab → the "press again to exit" toast path (`useAndroidBack.ts`).
- There is no way back to the onboarding slides from here.

6. Screen 5 (email branch only) — EmailAuthScreen

- **File:** `/Users/matan/Projects/soccer/src/screens/auth/EmailAuthScreen.tsx`
- **Shown when:** the user taps 'המשיך עם מייל' on SignIn (`SignInScreen.tsx:239`). Registered at `src/navigation/AuthStack.tsx:27` .
- **Mode:** `const [mode, setMode] = useState<'signIn' | 'signUp'>('signIn')` — `:45` . **The screen opens in SIGN-IN mode.** A brand-new user must first tap the toggle link at the bottom to reach sign-up.
- **Header:** `<ScreenHeader title={isSignUp ? he.emailAuthSignUpTitle : he.emailAuthSignInTitle} />` (`:170–172`). `ScreenHeader` renders a back chevron when `navigation.canGoBack()` (`src/components/ScreenHeader.tsx:46, :59–64` ; in RTL the back glyph is `chevron-forward` , `:61–62`) — so this is **the first screen in the whole flow with a visible back affordance**.
- `KeyboardAvoidingView` with `behavior='padding'` on iOS only (`:173–176`).

STRINGS, VERBATIM

Element	Value	he.ts	Rendered
Title (sign-in mode)	emailAuthSignInTitle: 'התחברות עם מייל'	:1813	:171
Title (sign-up mode)	emailAuthSignUpTitle: 'הרשמה עם מייל'	:1814	:171
Email label	emailAuthEmailLabel: 'כתובת מייל'	:1815	:178
Email placeholder	emailAuthEmailPlaceholder: 'name@example.com'	:1816	:182

Element	Value	he.ts	Rendered
Password label	emailAuthPasswordLabel: 'סיסמה'	:1817	:193
Password placeholder	emailAuthPasswordPlaceholder: 'לפחות 6 תווים'	:1818	:198
Confirm label (sign-up only)	emailAuthConfirmPasswordLabel: 'אימות סיסמה'	:1819	:226
Confirm placeholder	emailAuthConfirmPasswordPlaceholder: 'הקלד שוב את הסיסמה'	:1820	:231
Inline mismatch error	emailAuthPasswordMismatch: 'הסיסמאות לא תואמות'	:1821	:259
Forgot link (sign-in only)	emailAuthForgot: 'שכחת סיסמה?'	:1826	:266
CTA (sign-in)	emailAuthSignInCta: 'התחבר'	:1822	:284
CTA (sign-up)	emailAuthSignUpCta: 'הרשמה'	:1823	:284
Toggle → sign-up	emailAuthToggleToSignUp: 'אין לך חשבון? הרשמה'	:1824	:298
Toggle → sign-in	emailAuthToggleToSignIn: 'כבר יש לך חשבון? התחבר'	:1825	:298
Eye button ally	'הסתר סיסמה' / 'הצג סיסמה'	hardcoded	:212 and :249

CONTROLS

- **Email** TextInput (:179–191): keyboardType="email-address", autoCapitalize="none", autoComplete="email", contentType="emailAddress", textAlign="right".
- **Password** TextInput (:195–207): secureTextEntry={!showPassword}, autoComplete flips 'password-new' / 'password' by mode (:203).
- **Eye toggle** (:208–219 , and a duplicate at :245–256 for the confirm field) — one shared showPassword state (:49), positioned left: spacing.md (:338–339) i.e. the far side under RTL.
- **Confirm password field** — rendered only when isSignUp (:224–262).
- 'שכחת סיסמה?' — rendered only when !isSignUp (:264–268).
- **Primary CTA** (:270–287): disabled={!canSubmit}, dimmed to opacity: 0.5 when disabled (:276), <ActivityIndicator /> while busy (:280–281).
- **Mode toggle link** (:289–300): switchMode(...) (:64–70 , logs AnalyticsEvent.AuthModeSwitched) and clears the confirm field (:292).

VALIDATION (ALL CLIENT-SIDE, :37, :52–60)

- EMAIL_RE = /^[^s@]+@[^s@]+\.[^s@]+\$/ (:37)
- passwordOk = password.length >= 6 (:54)
- Sign-up additionally requires passwordsMatch && confirmPassword.length > 0 (:60)
- confirmPassword only shows once the user has typed something into confirm (:58) — deliberate, per the comment at :55–56 .
- **The CTA is disabled, not error-flagged**, until all of the above pass. There is no inline error for a bad email or a short password — only the disabled button.

EVERY ERROR STATE

Resolved in handleAuthError (:89–135):

Firestore code	Message	he.ts
auth/invalid-email	emailAuthInvalidEmail: 'כתובת מייל לא תקינה'	:1827
auth/weak-password	emailAuthWeakPassword: 'הסיסמה חייבת להכיל לפחות 6 תווים'	:1828
auth/email-already-in-use	emailAuthAlreadyInUse: 'המייל הזה כבר רשום. אם נרשמת עם Google/Apple – מייל וסיסמה – עבור להתחברות. אם נרשמת עם Google/Apple – מייל וסיסמה – חזור והשתמש בכפתור המתאים' with two actions: cancel: 'בטל' (he.ts:52) and emailAuthSwitchToSignIn: 'עבור להתחברות' (he.ts:1834)	:1832–1833 , dialog at :114–117
auth/wrong-password / auth/user-not-found / auth/invalid-credential	emailAuthWrongCredentials: 'מייל או סיסמה שגויים'	:1829
auth/too-many-requests	emailAuthTooManyAttempts: 'יותר מדי ניסיונות. נסה שוב'. 'בעוד כמה דקות'	:1830
auth/network-request-failed	signInNetworkError: 'אין חיבור לאינטרנט'	:1270
unknown	emailAuthGenericError: 'משהו השתבש, נסה שוב' + logError to the error panel	:1831 , :106 , :131
EmailRegisteredWith ProviderError google	emailAuthRegisteredWithGoogle: 'הכתובת הזו כבר רשומה דרך Google. התחבר עם Google.'	:1835 , thrown from src/firebase/auth.ts:181–190
EmailRegisteredWith ProviderError apple	emailAuthRegisteredWithApple: 'הכתובת הזו כבר רשומה דרך Apple. התחבר עם Apple.'	:1836

Password reset (onForgotPassword , :137–166): - No/invalid email typed → emailAuthResetNeedEmail: 'הקלד' (he.ts:1840), logged as PasswordResetRequested { result: 'blocked', reason: 'invalid_email' } (:139–142). - Success (and, deliberately, also user-not-found — anti-enumeration, :161) → title emailAuthResetSentTitle: 'נשלח מייל איפוס' (he.ts:1837), body emailAuthResetSentBody: (email) => \ \${email}. שלחנו קישור לאיפוס סיסמה אל \ (he.ts:1838–1839).

WHAT THE ACCOUNT LOOKS LIKE AFTER EMAIL SIGN-UP

userService.signUpWithEmail (src/services/userService.ts:306–341) writes name: '' (:322), onboardingCompleted: false (:326). A verification email is sent best-effort and **does not block usage** — src/firebase/auth.ts:171–173 , and the comment at userService.ts:303–304 . **There is no email-verification gate anywhere in the flow.**

Because name is empty, the email user must type their name on the next screen. The Google user does not.

BACKING OUT

Back chevron pops to SignIn. Android back pops the same way (rule 1 in `useAndroidBack.ts:11–12`). Nothing is persisted — returning to this screen resets to sign-in mode with empty fields.

7. The push-permission prompt — fires on top of the next screen

`src/navigation/RootNavigator.tsx:235–248` :

```
useEffect(() => {
  if (!currentUser) return;
  if (currentUser.isGuest) return; // :240
  notificationsService.requestAndRegisterPushToken(currentUser.id)
```

This effect runs the moment `currentUser` is set — which is the same render in which `RootNavigator` has already decided to show `PostSignInOnboardingScreen` (:267–269). So on a real device the **OS push-permission dialog appears over the "בוא נכיר" profile screen, immediately after the sign-in sheet closes**, before the user has typed anything.

Implementation `src/services/notificationsService.ts:634–696` : - Bails out in Expo Go / `storeClient` (:641–652). - `getPermissionsAsync()` (:671); only if not granted **and** `canAskAgain` does it call `requestPermissionsAsync()` (:673–675). - Logs `AnalyticsEvent.PushPermissionResult { granted, can_ask_again }` (:676–679). - Then `getFcmToken()` + `registerDeviceToken` (:685–687). - Guests are skipped entirely — rationale at `RootNavigator.tsx:237–240` : "an anonymous session shouldn't trigger the OS push-permission prompt (App Store 5.1.1 friction)". - `android.permissions` in `app.json` includes `"android.permission.POST_NOTIFICATIONS"` (so Android 13+ shows a runtime dialog). - There is **no pre-prompt / priming screen** explaining why notifications matter. The system dialog is the first and only ask. - Failure string used in the internal error panel: `requestAndRegisterPushToken: 'בקשת הרשאת התראות נכשלה'` (`src/services/errorLog.ts:185`).

`joryio.identify(user.uid, { email, name })` fires from the auth-state listener the first time a user appears — `src/firebase/auth.ts:466–474`, with the rationale at :450–455 (a day-one user used to spend the whole first session as an anonymous Joryio record).

8. Screen 6 — Post-sign-in profile (`PostSignInOnboardingScreen`)

- **File:** `/Users/matan/Projects/soccer/src/screens/onboarding/PostSignInOnboardingScreen.tsx`
- **Shown when:** `!isGuest && currentUser && !hasCompletedOnboarding` (`RootNavigator.tsx:267–269`), i.e. every newly-created Google / Apple / email account.
- **Skipped when:** guest (`isGuest`), or a returning account whose `/users/{uid}.onboardingCompleted === true`.
- **History:** the header comment (:1–6) records that this used to be **three** screens (welcome → how it works → profile) and was collapsed to one because "the user already saw the value pitch on the pre-sign-in slides; repeating it here just adds taps before the app actually starts working."
- **Layout:** blue gradient hero with rounded bottom corners, `HERO_GRADIENT = ['#1E3A8A', '#1E40AF', '#3B82F6']` (:32), 132dp live avatar preview pulled up onto the hero (:152–161), then a form card, then a bottom-pinned CTA bar (:221–235).

STRINGS, VERBATIM

Element	Value	he.ts	Rendered
Hero title	psoProfileTitle: 'בוא נכיר'	:2839	:142
Hero subtitle	psoWelcomeBody: '— מארגנים כדורגל שכוונתי בלי בלגן — 'הרשמה, ספסל, קבוצות, שוערים וטיימר'	:2808– 2809	:143
Name field label	profileName: 'שם'	:1847	:165
Name placeholder	profileNamePlaceholder: 'איך לקרוא לך?'	:1848	:168
Photo section label	profilePhotoLabel: 'תמונת השחקן'	:2810	:174
Upload button	profilePhotoUpload: 'העלאה מהגלריה'	:2811	:188
Upload button after a photo is set	profilePhotoChange: 'החלף תמונה'	:2812	:188
Avatar grid label	profileAvatarLabel: 'או בחר אווטאר'	:2813	:192
CTA	psoProfileSave: 'המשיך'	:2840	:233

Note `psoWelcomeBody` is a **value pitch repeated after sign-up**, and it is worded in a different register ('מארגנים' — impersonal plural) from the hero title above it ('בוא נכיר' — masculine singular).

The user's email is shown as a small centred caption when present: `{user?.email ? <Text style={styles.email}>{user.email}</Text> : null}` (:217).

CONTROLS

1. **Name** `InputField` (:164–172) — `maxLength={40}`, `icon="person-outline"`, `required` (:171). The `required` prop renders a **red asterisk** next to the label (`src/components/InputField.tsx:53–56`, :103–109). Pre-filled from `user?.name` (:40) — populated for Google (`userService.ts:183`) and usually for a first-time Apple authorization (`userService.ts:234`), **empty for email sign-up** (`userService.ts:322`).
2. 'העלאה מהגלריה' (:175–190) → `handlePickPhoto` (:63–96) → `pickAndUploadAvatar(user.id)` (`src/services/photoService.ts`). This is where the **photo-library permission dialog** fires: `ImagePicker.requestMediaLibraryPermissionsAsync()` at `src/services/photoService.ts:123`, then `launchImageLibraryAsync` at :131. iOS prompt copy comes from `app.json` → `expo.ios.infoPlist.NSPhotoLibraryUsageDescription: "Teamder מבקשת גישה לתמונות שלך כדי שתוכל להעלות אווטאר אישי או תמונת קאבר לקהילה שלך"` (a camera string also exists: `NSCameraUsageDescription: "Teamder מבקשת גישה למצלמה כדי לצלם אווטאר אישי בלי לעזוב את המשיך"`).
(`האפליקציה`).
3. **Avatar grid** (:193–215) — 24 procedurally-rendered avatars from `AVATARS` in `/Users/matan/Projects/soccer/src/data/avatars.ts:23–55` (coloured circle + emoji glyph: 🏠🏆🏆🏆, then skin-tone/hair/age variants 🧑🧒...🧒). Selection ring `avatarCellActive` (:336–338). One is **pre-selected at random** on mount: `user?.avatarId ?? (user ? pickRandomAvatarId() : undefined)` (:48–50), and the sign-in services also assign a random `avatarId` at doc creation (`userService.ts:185`, :236, :281, :324).
4. Picking a photo clears the avatar (:94); picking an avatar clears the photo and best-effort deletes the uploaded file (:99–106).

5. 'המשך' CTA (:222-234) — disabled={!canSave} where canSave = name.trim().length > 0 && !busy && !uploading (:61). Dimmed to opacity: 0.5 (:228). handleSave (:113-131) → completePostSignInOnboarding({ name, avatarId: photoUrl ? undefined : avatarId, photoUrl }) → userService.completeOnboarding → logEvent(ProfileCreated) + logEvent(OnboardingCompleted) (src/store/userStore.ts:253-258).

REQUIRED VS SKIPPABLE

- **A name is required.** There is no skip link, no "later", no back chevron, no header at all. This screen is a hard gate.
- Android hardware back: no stack to pop and not on a tab → "press again to exit" then the app closes. **Backing out of this screen means leaving the app with an account that is signed in but has onboardingCompleted: false** — the next launch lands right back here.
- Photo and avatar are both optional (an avatar is pre-selected anyway).

ERROR STATES

- Photo cancelled or permission denied → **silent**, no dialog. Only analytics: PhotoUploadAbandoned { source: 'onboarding', reason: 'permission_denied' | 'cancelled' } (:74-77). The comment at :69-72 says this is deliberate, per App Store guideline 5.1.1(iv) — never nag after a denial.
- Note: profilePhotoPermissionDenied: 'אין הרשאה לגישה לגלריה. אפשר לאשר בהגדרות הטלפון.' exists in he.ts:2814-2815 but is **not** shown on this screen.
- reason === 'network' → appAlert(he.error, he.profilePhotoUploadFailed) where profilePhotoUploadFailed: 'העלאת התמונה נכשלה. נסה שוב.' (he.ts:2816), at :84-85.
- reason === 'unavailable' → profilePhotoUnavailable: 'בחירת תמונה לא זמינה כרגע. בחר אוטאר.' (he.ts:2818-2819), at :86-87.
- Save failure → appAlert(he.error, he.signInFailed) — i.e. 'ההתחברות נכשלה. נסה שוב.', **a sign-in message shown for a profile-save failure** (:127). Analytics ProfileSaveFailed { source: 'post_signin_onboarding', code } (:123-126).

9. Screen 7 (effectively unreachable for new users) — ProfileSetupScreen

- **File:** /Users/matan/Projects/soccer/src/screens/auth/ProfileSetupScreen.tsx. Registered at src/navigation/AuthStack.tsx:26.
- **Shown when:** !isGuest && !profileComplete — but only **after** the !hasCompletedOnboarding check has already passed (RootNavigator.tsx:267 runs before :271). Since completePostSignInOnboarding can only be reached with a non-empty name (PostSignInOnboardingScreen.tsx:61), a newly-created account can never land here. It is reachable only by a grandfathered account that has onboardingCompleted === true **and** an empty name — e.g. a pre-existing user whose name was later cleared.
- **Strings:**
 - Title profileTitle: 'בוא נכיר' (he.ts:1846) — rendered :69. Identical copy to the post-sign-in hero.
 - Name label profileName: 'שם' (he.ts:1847) — :77
 - Placeholder profileNamePlaceholder: 'אֵיךְ לקרוא לך?' (he.ts:1848) — :80
 - CTA profileSave: 'שמור והמשך' (he.ts:1850) — :93
 - Unused here but adjacent in he.ts: profileNameRequired: 'שם הוא שדה חובה' (:1849)

- **Layout:** `ScreenContainer` → centred `PlayerIdentity` preview card (:71–73) → name `InputField` with `required` and `maxLength={40}` (:76–88) → bottom `Button` (:92–100). `returnKeyType="done"` submits (:84–87).
- **Errors** (:28–55): `RESERVED_NAME` → `officialNameTaken`: 'השם הזה שמור לחשבון הרשמי של Teamder.' (`he.ts:640–641`); `EMAIL_NAME` → `emailNameNotAllowed`: 'כתובת אימייל היא לא שם.' (`he.ts:642–643`); otherwise `profileSaveError`: 'לא הצלחנו לשמור את הפרטים. בדוק.' (`he.ts:189`). `Analytics ProfileSaveFailed { source: 'profile_setup', code }` (:41).
- **Note:** these two reserved-name errors are enforced on `ProfileSetupScreen` but the `PostSignInOnboardingScreen` save path shows only `he.signInFailed` (`PostSignInOnboardingScreen.tsx:127`) for the same underlying failures.
- **No back, no skip.** Hard gate, same as §8.

10. Overlays that can appear during any of the above

Mounted in `App.tsx` 's render tree (:920–1054) and therefore able to cover a first-run screen.

Overlay	Mount	Active condition	Copy
MockMod eBanner	<code>App.tsx</code> : 970	dev/mock only, renders nothing in prod (<code>App.tsx:962–963</code>)	—
Announc ementBa nner	<code>App.tsx</code> : 971	Remote Config <code>announcement_enabled</code> + non-empty <code>announcement_text</code> (<code>src/components/RemoteGates.tsx:35–67</code>)	server-authored <code>announcement_text</code> ; dismissible × (:61–63)
Force update modal	<code>App.tsx</code> : 1015– 1017	<code>splashDone</code> && <code>updateKind</code> === 'force'	title <code>updateForceTitle</code> : 'נדרש עדכון' (<code>he.ts:151</code>), body <code>updateForceBody</code> : 'יש גרסה חדשה לאפליקציה. חובה לעדכן כדי להמשיך' (<code>he.ts:152</code>), single CTA <code>updateNow</code> : 'עדכן עכשיו' (<code>he.ts:155</code>). Not dismissible — <code>onRequestClose={() => {}}</code> and backdrop press disabled (<code>src/components/UpdateModal.tsx:21, :25</code>). This can be the first interactive thing a brand-new installer sees , before the slides are even usable.
Optional update modal	<code>App.tsx</code> : 1018– 1032	<code>splashDone</code> && <code>updateKind</code> === 'optional'	<code>updateOptionalTitle</code> : 'גרסה חדשה זמינה' (<code>he.ts:153</code>), <code>updateOptionalBody</code> : 'יש גרסה חדשה זמינה לאפליקציה' (<code>he.ts:154</code>), buttons <code>updateLater</code> : 'אולי אחר כך' (<code>he.ts:156</code>) + <code>updateNow</code> : 'עדכן עכשיו'. Dismissal snoozes 24h via <code>OPTIONAL_UPDATE_SNOOZE_KEY</code> (<code>App.tsx:300–301, :1025–1028</code>).
Mainten anceGat e	<code>App.tsx</code> : 1036	Remote Config <code>maintenance_mode</code>	full-screen blocking; title 'בתחזוקה' hardcoded at <code>src/components/RemoteGates.tsx:28</code> ; default body 'אנחנו עורכים תחזוקה קצרה ונחזור עוד מעט.' at :17–18, overridable by <code>maintenance_message</code> .

Overlay	Mount	Active condition	Copy
CampaignGate	App.tsx: 1040	splashDone && updateKind === 'none' && !!currentUserId — no onboarding gate , so it can fire while the user is still on the post-sign-in profile screen. 1200ms delay after becoming active (src/components/CampaignGate.tsx:38–49).	Pulse-authored popupTitle / popupBody ; default button text 'הבנתי' (CampaignGate.tsx:80).
WhatsNewGate	App.tsx: 1044–1051	splashDone && updateKind === 'none' && !!currentUserId && onboardingComplete	On a fresh install seen is null , so resolveWhatsNew falls into the baseline === null branch and surfaces the current version's highlights (src/services/whatsNewService.ts:78–84). A brand-new user who has just finished signing up can therefore be shown a "what's new in this version" modal for a version they have never not had. The comment at :72–77 acknowledges the fresh-install case explicitly.
ScreenshotReportSheet	App.tsx: 982	testers only — if (!isTester) return; (src/components/ScreenshotReportSheet.tsx:88–91). Not part of a normal new user's run.	screenshotReportTitle: 'צילמת מסך – לדווח על?' (he.ts:1751)
InAppMessageHost (Joryio)	App.tsx: 986	inside the navigator, so Joryio in-app campaigns can render at any point	server-authored
BannerAd	in the tab bar, src/navigation/MainTabs.tsx:56–68	gated only by EXPO_PUBLIC_ADMOB_ENABLED (src/services/adsService.ts:123), the Pulse master switch, and RC banner_enabled (adsService.ts:516–519) — no new-user grace	An ad banner is present on the home screen from the account's first second. The app-open ad, by contrast, <i>does</i> have a new-user grace: Date.now() – opts.accountCreatedAt < rcNumber('app_open_new_user_grace_ms') (adsService.ts:442–443).

11. Screen 8 — the landing screen (MainTabs → "בית")

- **File:** /Users/matan/Projects/soccer/src/navigation/MainTabs.tsx , initialRouteName="ProfileTab" (:91), root screen Profile = /Users/matan/Projects/soccer/src/screens/tabs/ProfileScreen.tsx (src/navigation/ProfileStack.tsx:16, :132).
- Tab labels, right-to-left order (MainTabs.tsx:30–32 explains index 0 = rightmost under RTL):
- tabHome: 'בית' — he.ts:2887 , used MainTabs.tsx:129
- tabCommunities: 'מועדונים' — he.ts:1273 , used :137
- tabGame: 'מחזורים' — he.ts:2885 , used :150
- tabChat: 'צ'אטים' — he.ts:4049 , used :159

- **First thing a new account sees on this screen:** a time-based greeting (`greetingMorning`: 'בוקר טוב' `he.ts:3073`, `greetingNoon`: 'צהריים טובים' `:3074`, `greetingEvening`: 'ערב טוב' `:3075`, `greetingNight`: 'לילה טוב' `:3076`; used `ProfileScreen.tsx:787–790`), plus the **activation checklist** (`ProfileScreen.tsx:1366–1371`, rendered while `homeDataReady && !checklistComplete`).
- Checklist card (`/Users/matan/Projects/soccer/src/components/home/OnboardingChecklist.tsx:73–77`):
- `homeChecklistTitle`: 'בוא נתחיל' + a ✕ appended in JSX (`OnboardingChecklist.tsx:74–76`) — `he.ts:1990`
- `homeChecklistSubtitle`: 'כמה צעדים קטנים כדי להפיק את המקסימום' — `he.ts:1991`
- progress ring `${done}/${total}` (`:62`)
- The five steps (`ProfileScreen.tsx:641–710`), each tappable: 1. `homeStepPhoto`: 'הוספת תמונת פרופיל' (`he.ts:1992`, `ProfileScreen.tsx:644`) → `ProfileEdit`. Done when `user.photoUrl` — **an avatar does not count**, so a user who picked an avatar on the post-sign-in screen still sees this step undone. 2. `homeStepAvailability`: 'סמן מתי אתה פנוי' (`he.ts:1993`, `:657`) → `AvailabilityEdit` 3. `homeStepCommunity`: 'הצטרף או פתח מועדון' (`he.ts:1994`, `:670`) → `CommunitiesTab` 4. `homeStepGame`: 'הצטרף או צור מחזור ראשון' (`he.ts:1995`, `:683`) → `GameTab` 5. `homeStepInvite`: 'הבא חבר למגרש' (`he.ts:1996`, `:696`) → share sheet; done only when someone actually joined via the link (`:698–701`)
- `homeStepPosition`: 'בחר עמדה מועדפת' (`he.ts:1997`) exists but is **not** in the rendered list.
- Note the third repetition of the same phrase: 'בוא נכיר' (post-sign-in hero, `he.ts:2839`), 'בוא נכיר' (`ProfileSetup` title, `he.ts:1846`), 'בוא נתחיל' (checklist, `he.ts:1990`) and 'בואו נתחיל' (`SignIn` title, `he.ts:1793`).

12. The guest branch, and the wall it ends at

A guest reaches `MainTabs` in **one tap** from `SignIn` (`SignInScreen.tsx:256`, `RootNavigator.tsx:260`, `:279`). They have no `/users` doc (`userService.ts:56–60`) and no push token (`RootNavigator.tsx:240`).

The moment they try to do anything that needs an account, `ensureNotGuest`

(`/Users/matan/Projects/soccer/src/utils/guestGate.ts:39–63`) shows a dialog: - Title

`guestRegisterTitle`: 'נדרשת הרשמה' (`he.ts:1801`) - Body, context-specific: - default `guestRegisterBody`:

'?כדי להשתמש בתכונה הזו צריך חשבון. רוצה להירשם עכשיו' (`he.ts:1802`) - joining a game

`guestRegisterJoinGame`: '?כדי להירשם למחזור צריך חשבון. רוצה להירשם עכשיו' (`he.ts:1803`) — used at

`src/screens/games/MatchDetailsScreen.tsx:907` - joining a club `guestRegisterJoinCommunity`: 'כדי

'?להצטרף למועדון צריך חשבון. רוצה להירשם עכשיו' (`he.ts:1804`) - messaging an admin

`guestRegisterChatAdmin`: '?כדי לשלוח הודעה למנהל צריך חשבון. רוצה להירשם עכשיו' (`he.ts:1805`) -

creating `guestRegisterCreate`: '?כדי ליצור צריך חשבון. רוצה להירשם עכשיו' (`he.ts:1806`) - chat

`guestRegisterChat`: '?כדי להשתמש בצ'אט צריך חשבון. רוצה להירשם עכשיו' (`he.ts:1807`) - Buttons:

`cancel`: 'בטל' (`he.ts:52`) and `guestRegisterCta`: 'הרשמה' (`he.ts:1808`) - Choosing 'הרשמה' stashes the return target then calls `signOut()` (`guestGate.ts:51–58`), which **tears down the whole navigator and drops the user back on SignIn** — they then re-run §5 → §8. The stashed target is replayed by the pending-invite

consumer in `RootNavigator.tsx:63–166` once the new account is ready. - The profile tab also shows a standing guest card: `guestProfileTitle`: 'הפרופיל שלך מחכה' (`he.ts:1809`) + `guestProfileBody`: 'אתה גולש'

(`he.ts:1810–1811`) + 'כאורח. הירשם כדי לשמור מחזורים, להצטרף למועדונים ולבנות פרופיל שחקן' (`he.ts:1810–1811`) +

'הרשמה' button — `ProfileScreen.tsx:1116–1119`.

13. Deep-link / install-attribution entry points that alter the first run

Cold start, in strict priority order (App.tsx:537-577): 1. Linking.getInitialURL() → handleCold (:539-541 , handler :436-458) 2. existing stash in storage — if present, **stop**, skip 3 and 4 (:545-547) 3. Android Play Install Referrer — consumeInstallReferrerIfFresh() (:552-553 , src/services/installReferrerService.ts) 4. iOS clipboard deferred deep link — consumeClipboardInviteIfFresh() (:568-569 , src/services/clipboardInviteService.ts). Rationale at App.tsx:562-567 : Apple has no install-referrer API, so the landing page copies the invite URL to the clipboard.

An invite-opened launch calls adsService.noteIntentfulOpen() to suppress the next app-open ad (App.tsx:452, :488). A campaign link footy://open/<where> with a role query param records the role via recordRole (App.tsx:444, :478 → src/services/roleService.ts:23-30 → joryio.setAttributes({ user_type: role })). The pending-destination consumer deliberately **does not wait on auth** — comment at App.tsx:583-586 : "the role picker shows on a first run, and the tab it opens is reachable before a profile is complete."

The invite target itself is only navigated to once the user is signed in, profile-complete **and** onboarded (RootNavigator.tsx:65) — so an invited user still walks the full slides → sign-in → profile flow before reaching the game or club they tapped.

URL schemes / domains (app.json): scheme: ["footy", "teamder", "com.studiogameslime.soccerapp"] ; iOS associatedDomains: ["applinks:teamderfc.web.app"] ; Android verified App Links on https://teamderfc.web.app with pathPrefix /session , /team , /app . **There is no custom domain** — the only host is the Firebase Hosting default teamderfc.web.app , and it appears verbatim in the app's iOS entitlement and Android intent filter.

14. Tap-by-tap counts

"Tap" = one deliberate user action. System-UI taps (Google chooser, OS permission dialogs) are marked **[OS]** and counted separately so they can be excluded if the consultant prefers.

BRANCH A — GOOGLE, NEW USER, NO DEEP LINK (THE LIKELIEST ANDROID PATH)

#	Screen	Action
—	Native splash	wait (~instant)
—	Animated splash	wait ≥1.4s + 0.32s fade
1	Onboarding slide 1	tap 'הבא' (or swipe)
2	Onboarding slide 2	tap 'הבא' (or swipe)
3	Onboarding slide 3	tap 'המשך'
4	SignIn	tap 'Google עם המשך'
[OS]	Google account chooser	tap the account (+ a Play Services update dialog if stale, auth.ts:93-95)
[OS]	Push permission dialog (fires over the next screen)	tap Allow / Don't allow
5	'בוא נכיר'	tap 'המשך' (name already pre-filled from Google)

#	Screen	Action
—	Animated splash again	wait for <code>groupHydrated</code> (<code>RootNavigator.tsx:274</code>)
—	'בית'	arrived

5 in-app taps, 2 OS taps, 0 text entries, 6 distinct screens (2 splashes + 3 slides as one screen + sign-in + profile + home). Minimum path if the user taps 'דלג' on slide 1: **3 in-app taps** (דלג → Google → המשך), 2 OS taps, 0 text entries.

BRANCH B — EMAIL SIGN-UP, NEW USER

#	Screen	Action
1-3	Onboarding	'המשך', 'הבא', 'הבא'
4	SignIn	tap 'המשך עם מיל'
5	EmailAuth (opens in sign-in mode, :45)	tap 'אין לך חשבון? הרשמה'
T1		type email
T2		type password (≥6)
T3		re-type password
6		tap 'הרשמה'
[OS]		push permission dialog
T4	'בוא נכיר'	type name (empty — <code>userService.ts:322</code>)
7		tap 'המשך'
—		splash → 'בית'

7 in-app taps, 1 OS tap, 4 text entries, 7 distinct screens. Add 1 tap if the user also picks an avatar, 2+ taps and an [OS] photo-permission dialog if they upload a photo. Minimum with 'דלג': **5 in-app taps**, 4 text entries.

BRANCH C — APPLE, NEW USER, IOS

Same shape as A. 'Apple' 'המשך עם' at `SignInScreen.tsx:229` (iOS only, :212). The Apple sheet itself typically costs 1-2 [OS] interactions (share/hide email choice, then Face ID / Touch ID / password) — **not verified on device**. Name arrives from `fullName` on the *first* authorization only (`userService.ts:230-234`), so a re-authorizing user may land on 'בוא נכיר' with an empty name and need 1 text entry. **5 in-app taps, 2-3 OS taps, 0-1 text entries.**

BRANCH D — GUEST

#	Screen	Action
1-3	Onboarding	'המשך', 'הבא', 'הבא'
4	SignIn	tap 'המשך כאורח'
—		straight to 'בית'

4 in-app taps, 0 OS taps, 0 text entries, 4 screens. No push prompt (`RootNavigator.tsx:240`). But the account is not usable: the first attempt to join a game triggers the §12 wall, whose "register" path **signs them out** and restarts at SignIn — so the real cost of the guest path for a converting user is 4 taps + the dialog + the whole of

Branch A or B again.

BRANCH E — RETURNING USER, ALREADY SIGNED IN

Splash → (optional `groupHydrated splash`) → 'בית'. **0 taps**. A "מה חדש" modal may appear after a version bump (`App.tsx:1044-1051`).

BRANCH F — RETURNING USER AFTER SIGN-OUT, SAME INSTALL

`onboardingDone` survives sign-out (`src/store/userStore.ts:164-187` never touches it; key at `src/services/storage.ts:7`), so the slides are skipped: Splash → SignIn → (provider) → 'בית'. **1 in-app tap + 1 OS tap**. `onboardingCompleted` is already `true` on their `/users` doc, so 'בוא נכיר' is skipped too.

SCREEN COUNT SUMMARY

Distinct full screens a new Google user meets: **6** (native splash, animated splash, onboarding carousel, sign-in, post-sign-in profile, home) — 7 counting the second splash appearance separately. A new email user meets **7** (adds EmailAuth). A guest meets **4**.

15. Branches I did NOT check

Stated explicitly, per the brief: - **Nothing was run on a device or emulator**. Every claim above is read from source. The exact rendering, the real ordering of the OS push dialog against the profile screen's first paint, and the exact Apple/Google system-sheet copy are **not** verified on hardware. - I did not verify that the Play Install Referrer path or the iOS clipboard deferred-link path actually fire in a real store install — only that the code calls them and in what order (`App.tsx:552-576`). - I did not read the landing pages (`public/index.html`, `public/get.html`, `public/invite.html`, `public/c/`, `public/downloads/`) — out of this surface. - I did not enumerate `MainTabs` beyond the landing screen and the activation checklist. - I did not check mock mode (`USE MOCK DATA`) behaviour beyond noting that every auth path short-circuits to `mockCurrentUser` (`userService.ts:168-172`, `:215-218`, `:264-268`, `:307-311`), which means **mock QA does not exercise any of the real first-run auth branches**. - I did not verify the Remote Config defaults for `maintenance_mode`, `announcement_enabled`, `banner_enabled`, or `app_open_new_user_grace_ms` — only the code that reads them.

Open questions from this section

Things this reader could not settle from the code, listed rather than assumed.

- Real device ordering: does the OS push-permission dialog actually land on top of the 'בוא נכיר' screen, or does it appear a beat earlier/later? The effect at `RootNavigator.tsx:235-248` fires on the same render that chooses `PostSignInOnboardingScreen`, but only a device run proves the visual ordering.
- Exact copy and tap count of the Google account chooser and the Apple Sign-In sheet — system UI, not in our source. Also whether the Android Play Services update dialog (`auth.ts:93-95`) ever fires in practice on current devices.
- Does Apple's `fullName` actually arrive on a first authorization in the production build, or do iOS users land on 'בוא נכיר' with an empty name and an extra text entry? `userService.ts:230-234` hedges on this.
- On a genuine fresh install, does the 'מה חדש' modal actually appear right after sign-up? The fresh-install branch at `src/services/whatsNewService.ts:78-84` says it will surface the current version's highlights, but it also depends on the live `appConfig` `whatsNew` doc having items for version 1.1.10.

- Live Remote Config values for `maintenance_mode`, `announcement_enabled`, `banner_enabled` and `app_open_new_user_grace_ms` — these decide whether a new user sees an ad banner on the home screen in their first seconds and whether any blocking overlay fires.
- Whether the Play Install Referrer (Android) and clipboard deferred-link (iOS) paths at `App.tsx:552-576` actually recover an invite on a real store install — only the call order is verifiable from source.
- Is the intro carousel intended to be unreachable forever after the first skip/complete? The `AsyncStorage` flag `'footy.onboarding.done'` survives sign-out and account deletion on the same install, and there is no 'show intro again' anywhere.
- Whether the optional-update or force-update modal is currently armed for version 1.1.10 — a force modal would be the first interactive element a brand-new installer meets, before the slides.

5.2 · Acquisition and install — every way in

0. Scope & method

Everything below was read from source in `/Users/matan/Projects/soccer`. Nothing was run on a device or fetched from the live web. Where behaviour cannot be determined from code alone it is listed in **Open questions**, not guessed at.

Hosting: **two** Firebase Hosting sites with byte-identical config — `teamderfc` (`firebase.json:11`) and `soccer-app-52b6b` (`firebase.json:49`). There is **no custom domain**; every public URL a user ever sees is `https://teamderfc.web.app/...`. The second origin (`https://soccer-app-52b6b.web.app`) serves all the same pages but is **not** in the app's `HOSTING_DOMAINS` `allow-list` (`src/services/deepLinkService.ts:36-41`), so any link on that origin parses to `null` and loses both deep-link routing and attribution. Nothing in the repo generates such a link, but the origin is live and indexable.

Store identifiers, from source: - Android package `com.studiogameslime.soccerapp` — `app.json` (`expo.android.package`), Play URL `https://play.google.com/store/apps/details?id=com.studiogameslime.soccerapp` (`src/services/updateService.ts:9,14`). - iOS App Store id `6775178022` — `src/services/updateService.ts:12`, `eas.json` (`submit.internal.ios.ascAppId`), web URL `https://apps.apple.com/app/id6775178022`. - Both are also duplicated as Remote Config defaults: `store_url_ios` / `store_url_android` at `src/services/remoteConfigService.ts:54-56`.

1. The route table (what exists on the web)

`firebase.json:21-38`, site `teamderfc`:

URL	Served by	Template	OG injected server-side?
/	static	<code>public/index.html</code>	no (static tags)
/get	static	<code>public/get.html</code>	no
/app	rewrite → <code>/invite.html</code> (<code>firebase.json:22</code>)	<code>public/invite.html</code>	no

URL	Served by	Template	OG injected server-side?
/go	rewrite → /invite.html (firebase.json:23)	public/invite.html	no
/session/**	rewrite → /invite.html (firebase.json:21)	public/invite.html	no
/team/**	CF serveCommunityPage (firebase.json:27–29)	functions/templates/ invite.html	yes (functions/src/in dex.ts:10619– 10678)
/c/**	CF serveCommunityPage (firebase.json:31–33)	functions/templates/ community.html	yes
/i/**	CF serveInviteCode (firebase.json:35– 37)	functions/templates/ invite.html	yes (functions/src/in dex.ts:10687– 10825)
/invite- preview	CF getInvitePreview (firebase.json:25)	JSON API	n/a
/track- click	CF trackLinkClick (firebase.json:24)	204 beacon	n/a
/privacy , /delete- account	static (cleanUrls:true , firebase.json:18)	public/privacy.html , public/delete- account.html	no
/downloads/ teamder- 0.2.5.aab	static	a 0.2.5-era Android App Bundle sitting in public/downloads/	n/a
/terms.html	does not exist — public/ contains only delete- account.html , get.html , index.html , invite.html , privacy.html	—	—

functions/templates/invite.html is byte-identical to public/invite.html (verified by diff), and functions/templates/community.html is byte-identical to public/c/index.html. Per project_ios_invite_attribution memory, the templates are **generated** from public/ by functions/scripts/copy-template.js at npm run build, and /i/ links only pick up an edit after firebase deploy --only functions:serveInviteCode — a hosting-only deploy does not update them.

2. Path-by-path, tap by tap

PATH A — WHATSAPP SHORT INVITE LINK /I/<CODE> (THE MODERN DEFAULT; EVERY IN-APP SHARE PRODUCES THIS)

All six in-app share entry points call createShortInviteUrl (src/services/inviteLinkService.ts:35–66), which writes inviteLinks/{code} and returns https://teamderfc.web.app/i/<7-char base62 code> (inviteLinkService.ts:16,22,58), with ?invitedBy=<uid> appended when an inviter is known (inviteLinkService.ts:59–61).

A-Android, app not installed: 1. Person taps the link inside WhatsApp → WhatsApp's in-app browser (Android WebView) loads `/i/<code>. serveInviteCode` reads `inviteLinks/{code}`, increments `clicks` and `inviteClicks/{inviter}` (`functions/src/index.ts:10712-10733`), injects `window.__INVITE__= {type,id,invitedBy}` before the page's inline script (`index.ts:10804-10809`), and injects the situation-aware OG block (`index.ts:10744-10799`). 2. Page fires `/invite-preview?code=...` (`public/invite.html:244`) and repaints hero copy + a context card with date / time / field / club / free-spots (`invite.html:246`). 3. Person taps `הצטרף למשחק / Teamder` / `הצטרף ל-Teamder` / `הורד את Teamder` (all three CTAs — header `hdrBtn`, hero `ctaBtn`, sticky `stickyBtn` — bound to the same `primary()` at `invite.html:232`). 4. `primary()` assigns `location.href = 'footy://session/<id>?invitedBy=...'` and starts a **1500 ms** fallback timer to the Play URL (`invite.html:231`). App absent → the WebView shows a scheme error for ~1.5 s, then redirects to Play with `&referrer=invite_session-<id>_by-<uid>` (`invite.html:229`). 5. Play Store → **פתח** → **התקן**.

Taps: 4 (link, CTA, Install, Open) plus a ~1.5 s dead interval on step 4. Note `/c/index.html:1046-1057` uses the correct Chrome `intent://...#Intent;scheme=footy;package=...;S.browser_fallback_url=...;end` pattern; `invite.html` does **not** — it only has the raw-scheme-plus-timeout dance.

A-iOS, app not installed: 1-3 as above. 4. `primary()` detects iOS (`invite.html:230`), writes `clipUrl()` — `https://teamderfc.web.app/session/<id>?invitedBy=...` (`invite.html:219`) — to the clipboard, then goes **straight** to `https://apps.apple.com/app/id6775178022`. The inline comment at `invite.html:231` documents why the `footy://` attempt is deliberately skipped on iOS (Safari renders a visible "cannot open page" error for an unhandled scheme). 5. App Store → **קבל** (+ Face ID / password) → 6. **פתח**. First launch: `consumeClipboardInviteIfFresh` (`src/services/clipboardInviteService.ts:38-94`) gates on `Clipboard.hasUrlAsync()` (no prompt) then `getStringAsync()`, which raises the iOS system "Allow Paste?" prompt — a 5th interaction the person must approve or attribution is lost.

Taps: 4-5 + one system paste prompt.

PATH B — LONG INVITE LINKS /SESSION/<ID> AND /TEAM/<ID>

Built by `deepLinkService.buildInviteUrl` (`src/services/deepLinkService.ts:203-213`) — `https://teamderfc.web.app/session/<id>?invitedBy=<uid>`. These are only used as the `fallbackLong` when the short-code write fails (`inviteLinkService.ts:62-65`), so in practice they appear only offline / rules-denied. Same tap count as Path A. Difference: `/session/**` is a **static** rewrite (`firebase.json:21`) with **no OG injection**, so its WhatsApp preview card is the generic `Teamder • מארגנים כדורגל בלי כאב ראש` (`public/invite.html:9`). `/team/**` goes through `serveCommunityPage` and does get the club's name, description and cover as OG (`functions/src/index.ts:10640-10650`, `10531-10554`).

PATH C — /GET (THE "TRY MY APP" MARKETING LINK)

`public/get.html:3-7` documents the intent: Facebook-group posts. On Android it hides the iOS button and `location.replace(ANDROID)`; on iOS the reverse (`get.html:59-65`). Desktop shows both and does not redirect (`get.html:66`). 1. Tap link → auto-redirect to the store (0 extra taps). 2. Install. 3. Open. **Taps: 3** — the cheapest path. **But:** the Android URL at `get.html:52` has **no** `&referrer=`, and there is no clipboard write. A `/get` install is 100 % unattributed on both platforms.

PATH D — THE CLUB SHOWCASE /C/<GROUPID>

`public/c/index.html`. On mobile it *auto-attempts* the deep link: Android Chrome via `intent://team/<id>#Intent;scheme=footy;package=com.studiogameslime.soccerapp;S.browser_fallback_url=<this page>;end` (`c/index.html:1046-1057`); everything else via `location.href='footy://team/'+id` (`c/index.html:1062`). Deliberately no store auto-redirect (comment at `c/index.html:1025-1032`) — the

showcase renders and the person reads it first. Store buttons: `heroDownload ( c/index.html:845 , label הורדת )` and `finalDownload ( c/index.html:939 , label Teamder את הורד )`, both re-pointed at `PLAY_URL + '&referrer=invite_team_<id>'` on Android or bare `APP_STORE_URL` on iOS (`c/index.html:1069-1081`). **Taps: 3-4.** iOS installs from `/c/` carry **no attribution at all** — no referrer equivalent and no clipboard write anywhere in this file.

PATH E – /APP AND /GO (GENERIC APP INVITE + PAID/AD LINKS)

buildAppInviteUrl → `https://teamderfc.web.app/app?invitedBy=<uid> (deepLinkService.ts:221-224)`.
`/go` is the dedicated acquisition path, both rewritten to the same static `invite.html` (`firebase.json:22-23`).
`invite.html` treats both as `isApp` (`invite.html:211`) and sets the Play referrer to `invite_app /`
`invite_app_by_<uid>` (`invite.html:229`). Ad links add `?b=<base64url source> / ?s= / ?c= / ?l=`
`<linkId> / ?g=<gameId>`; the page decodes `b` (`invite.html:206-208`), fires the `/track-click` beacon
(`invite.html:209`), and switches the Play referrer to a UTM querystring `utm_source=...&utm_campaign=...&g=...`
`&l=...` (`invite.html:229`). `parseReferrerInvite` reads that form back
(`src/services/installReferrerService.ts:94-111`) and `parseInviteUrl` reads it off a live URL
(`deepLinkService.ts:116-134`). The `b` token is explicitly described as the inverse of Pulse's
`encodeSourceToken` (`deepLinkService.ts:52-56`); **the Pulse link-builder itself is in `~/Projects/pulse`, not**
this repo. Taps: 4 (Android) / 4-5 (iOS), same dance as Path A.

PATH F – ORGANIC STORE SEARCH / BROWSE

No repo artefact. Play listing and App Store listing are configured outside the code; the only listing copy in-repo is `app-store-metadata.md`, which is a **paste-source draft, not the live listing** — and it still points at `https://teamder.app/privacy.html` and `https://teamder.app` (`app-store-metadata.md:178-179`), a domain that does not exist. Its "What's New" block is still the v1.0 launch text (`app-store-metadata.md:140-169`) while `app.json` is at version `1.1.10`. **Taps: 3** (search, Install, Open) once the person already knows the name.

PATH G – ROOT DOMAIN [HTTPS://TEAMDERFC.WEB.APP/](https://TEAMDERFC.WEB.APP/)

public/index.html is a **closed-beta recruitment page**. It has **zero store links**. The only CTAs are: -
 public/index.html:426-431 → anchor to #beta , label הצטרפו כבודקי בטא - public/index.html:563-568
 → https://wa.me/972546986121?text=... pre-filled Teamder.\n של המייל שלי :
 label שלחו את המייל בוואטסאפ - public/index.html:569-575 → mailto:studiogameslime@gmail.com?
 subject=לפטא הצטרפות Teamder&body=למייל שלי , label או שליחה במייל

The body copy at `public/index.html:556-561`:

שלחו לי את כתובת המייל שלכם (זו) / רוצה להיות מהראשונים לנסות? / בטא פתוחה • מקומות מוגבלים
שמחברת לחשבון Google של המכשיר). אוסיף אתכם לרשימת הבודקים ותקבלו קישור להורדה.

Taps to install from the root domain: undefined — there is no install path. Anyone who types the brand into a browser, or clicks the [לדף הבית](#) link on the community 404 state (`public/c/index.html:793`), lands here and is asked to email their address to join a beta of an app that is live in both stores.

PATH H — /DOWNLOADS/TEAMDER-0.2.5.AAB

A publicly-served 0.2.5 Android App Bundle. `.aab` is not sideloadable by a phone; tapping it downloads a file the OS cannot install. It is unreferenced by any page but reachable and crawlable.

PATH I — CLUB JOIN CODE (NOT AN INSTALL PATH)

`groupsSearchByCode: 'או הצטרף בעזרת קוד הזמנה' (src/i18n/he.ts:1255)`, `groupJoinCodeLabel: 'קוד הזמנה' (he.ts:2871)`, consumed by `groupService.requestJoinByCode` (`src/services/groupService.ts:722`) and by the first-run wizard (`src/services/onboardingService.ts:80–97`). This is post-install club discovery and carries **no** `invitedBy` — `applyInviteAttributionIfFresh` only reads the stored `PendingInvite` (`src/services/userService.ts:908–940`). A person who installs organically and types a friend's club code credits nobody.

3. The exact share text the app generates

Six call sites, all routed through `createShortInviteUrl`. Full inventory:

(a) **Game / match share — the one actually reachable.** `src/screens/games/MatchDetailsScreen.tsx:1652–1688` (`handleShare`), wired to the share icon in the match header (`MatchDetailsScreen.tsx:2702`, `src/components/match/MatchStadiumHero.tsx:132–148`) and to the bottom CTA in two roster states (`MatchDetailsScreen.tsx:1954`, `1970`). Builder: `he.sessionShareWhatsappBody` (`src/i18n/he.ts:302–316`):

```
{title} – מחפשים שחקנים!  
  
{when}  
  
{field} ← only when game.fieldName is set  
only when missing > 0 ← חסרים עוד {missing} שחקנים  
  
הצטרפו כאן  
  
{link}
```

Verbatim source strings: ``{args.title} – מחפשים שחקנים!` (he.ts:309)`, ``{args.field.trim()}` (he.ts:310)`, ``שחקנים {args.missing}$ חסרים עוד` (he.ts:312)`, ``הצטרפו כאן` (he.ts:314)`. Share sheet `title` is the raw game title (`MatchDetailsScreen.tsx:1677`).

(b) **Club / community share.** `src/screens/communities/CommunityDetailsScreen.tsx:453–488` (`handleInvite`), reached from the full-width CTA `CommunityShareInviteCta` (`CommunityDetailsScreen.tsx:1168`, label `communityMenuShareInvite: 'שתף הזמנה למועדון' — he.ts:1729`) and from the post-creation sheet (`CommunityDetailsScreen.tsx:1269–1277`). Builder `he.communityInviteShareBody` (`he.ts:1563–1573`):

```
Teamder-ב {name} הוזמנת להצטרף למועדון  
  
{description} ← only when set  
  
{link}
```

Verbatim: ``Teamder-ב {args.name}$ הוזמנת להצטרף למועדון` (he.ts:1568)`. Subject: `inviteShareSubject: 'הצטרף למועדון הכדורגל שלנו' (he.ts:1770)`.

The sheet that pushes this share says (he.ts:469-473): - inviteSheetTitle: (club) => \${club} מוכן. עכשיו צריך
מועדון בלי חברים לא עושה כלום. שלחו את הקישור בוואטסאפ — מי שילחץ עליו מצטרף ישירות, בלי' .אנשים
` - inviteSheetBody: 'הזמינו חברים' , inviteSheetLater: 'אחר כך' , inviteSheetCta: 'חיפושים'

is not what happens: a `team invite` navigates a non-member to `CommunityDetailsPublic` (`src/navigation/navigationRef.ts:134-138`), a preview screen from which they must still *request* to join.

(c) **App invite from the profile card.** `src/screens/tabs/ProfileScreen.tsx:924-946` (`handleShareInvite`), reached from the always-visible bottom CTA (`ProfileScreen.tsx:1380-1393` , label `profileInviteFriendsCta`: `'הזמן חברים לאפליקציה' — he.ts:3020`) and from the activation-checklist "invite" step (`ProfileScreen.tsx:695-708` , label `he.homeStepInvite`). Builder `he.profileInviteShareBody` (`he.ts:3021-3022`):

 **Teamder** אני משחק כדורגל בעזרת אפליקציית

תוריד גם אתה ובוא לשחק:

{link}

(d) **Friends-tab empty state.** `src/screens/profile/FriendsScreen.tsx:307–326` — same subject + same `profileInviteShareBody`. Surrounding copy: `friendsEmptyCtaTitle: 'בנה לעצמך רשימת חברים'` (`he.ts:1661`), `friendsEmptyCtaBody: 'הזמן חברים מרשימת אנשי הקשר שלך – תוכל להזמין אותם ישירות.'` (`he.ts:1662`), `friendsEmptyCtaButton: 'הזמן חברים לאפליקציה'` (`he.ts:1663`).

(e) **First-run wizard share step.** `src/services/onboardingService.ts:121–132` (`shareInvite`) — same subject + `profileInviteShareBody`, but the `url` handed in is a **club** link built by `clubInviteUrl` (`onboardingService.ts:49–63`), so a message that reads "download the app" carries a club-join URL. The wizard HTML itself is a Joryio in-app message authored outside this repo.

(f) **Joryio in-app message share action.** `src/components/joryio/InAppMessageHost.tsx:188-202` — same pair, url from `vars.inviteUrl`.

Dead / unreachable share text: - `he.sessionInviteShareBody` (`he.ts:296–297`) — הוזמנת למחזור

`n{link}\`: `Teamder`  `\nב-` — only referenced by `handleInvitePlayers`

(`MatchDetailsScreen.tsx:1462–1496`), and `handleInvitePlayers` has **no** `onPress` **anywhere** (grep for it returns the definition only). Its `game.visibility !== 'public'` guard (`MatchDetailsScreen.tsx:1468`) therefore never runs — the live `handleShare` has **no visibility check at all** (`MatchDetailsScreen.tsx:1653`), so a club-only game is shareable to the open internet. - `he.inviteShareBody` (`he.ts:1771–1772`) — הצטרף

`{link}`: `\nשם המועדון`  `\nבאפליקציה` — למועדון הכדורגל שלנו

zero call sites. - `he.sessionActionShareLink`: 'שיתוף קישור' (`he.ts:293`) and `he.sessionShareWhatsapp`: 'שתף בוואטסאפ' (`he.ts:298`) and `he.inviteShareTitle`: 'הזמן שחקנים' (`he.ts:1769`) — no call sites.

Non-invite share: teams export to WhatsApp, MatchDetailsScreen.tsx:2450–2473 , plain text
`{game.title}\n\n{colour-dot} {team name}\n\n{first names}` — carries **no link at all**, so an organiser posting the split into WhatsApp generates zero acquisition surface.

4. Attribution — where `invitedBy` survives and where it dies

Three independent recovery mechanisms, plus the write:

1. **Live deep link** — `parseInviteUrl` reads `?invitedBy=` (`deepLinkService.ts:110–114` , `184–188`), `App.tsx:449–457` stashes it.
2. **Android, Play Install Referrer** — `invite.html:229` appends `&referrer=invite_<type>_<id>_by_<uid>` ; `parseReferrerInvite` (`installReferrerService.ts:65–113`) reads three formats: `invite_app_by_<uid>` (line 68), `invite_(session|team)_<id>_by_<uid>` (line 76), legacy `invite_(session|team)_<id>` (line 84), plus the UTM form (line 94). Consumed once per install behind `installReferrerConsumed` (`installReferrerService.ts:146`).
3. **iOS, clipboard deferred deep link** — `invite.html:231` writes `clipUrl()` on the CTA tap; `clipboardInviteService.ts:38–94` reads it once on first launch, behind Remote Config `feature_ios_clipboard_invite` (default `true` , `remoteConfigService.ts:38`).

The write: `applyInviteAttributionIfFresh` (`src/services/userService.ts:908–940`), called from four fresh-user creation points (`userService.ts:155` , `208` , `252` , `339`). Hard fails: mock mode, no pending invite, no `invitedBy` , self-invite, and a **user doc that already has** `invitedBy` (`userService.ts:920`). Writes `invitedBy` , `invitedByType` , `invitedByTargetId` (`'app'` placeholder for generic invites, `userService.ts:925`) and a server-stamped `invitedAt` .

Where attribution is LOST, plainly:

Situation	Why
Any install via <code>/get</code>	No <code>&referrer=</code> on the Android URL (<code>get.html:52,61</code>); no clipboard write on iOS (<code>get.html:64</code>). Both platforms: nothing.
Any install via <code>/c/<id></code> on iOS	<code>c/index.html:1075</code> sends iOS straight to the bare <code>APP_STORE_URL</code> ; that file contains no clipboard write. Android is fine (<code>invite_team_<id></code> , <code>c/index.html:1069</code>) but has no inviter — the referrer is the club, never <code>_by_<uid></code> .
Any install via the root <code>/</code>	There is no install path at all.
iOS, person taps the CTA before <code>/invite-preview</code> resolves AND the CF didn't inject <code>__INVITE__</code>	<code>retarget()</code> (<code>invite.html:223–226</code>) hasn't run, so <code>pathPart</code> is still <code>'app'</code> and <code>clipUrl()</code> yields <code>/app?invitedBy=...</code> . The inviter survives; the game/club target is lost .
iOS, person declines the "Allow Paste?" prompt	<code>getStringAsync()</code> returns nothing usable; the <code>finally</code> block latches <code>clipboardInviteConsumed</code> anyway (<code>clipboardInviteService.ts:85–92</code>) — one shot, gone forever.
iOS, the CTA's <code>navigator.clipboard.writeText</code> throws	Swallowed silently (<code>invite.html:231</code> <code>try{...}catch(_){}</code>) — the person still reaches the App Store, unattributed, with no signal anywhere.
Any install where the person opens the store directly instead of tapping the page CTA	The referrer / clipboard is only set by <code>primary()</code> . Scrolling to the bottom and tapping the Google Play / App Store buttons (<code>invite.html:187–188</code>) uses <code>storeAndroid</code> 's href — which <i>does</i> carry the referrer after <code>invite.html:229 / 226</code> — but the App Store anchor at <code>invite.html:188</code> is a static href with no clipboard write , so an iOS visitor who uses that button instead of the CTA is unattributed.
A user who already has <code>invitedBy</code> set	Never overwritten (<code>userService.ts:920</code>) — correct, but means a re-install or a second invite is never re-credited.
Join by club code	Never touches <code>PendingInvite</code> .

Situation	Why
<code>/i/</code> with neither <code>__INVITE__</code> injection nor <code>?invitedBy=</code>	<code>type</code> stays <code>null</code> , <code>valid</code> is <code>false</code> (<code>invite.html:211</code>), <code>storeHref</code> gets no referrer at all (<code>invite.html:229</code> is guarded by <code>if(valid)</code>).
A link on <code>soccer-app-52b6b.web.app</code>	Not in <code>HOSTING_DOMAINS</code> (<code>deepLinkService.ts:36-41</code>) → <code>parseInviteUrl</code> returns <code>null</code> .

Click counting is inflated: `serveInviteCode` increments `inviteLinks/{code}.clicks` and `inviteClicks/{inviter}.clicks` on **every** **GET** (`functions/src/index.ts:10712-10733`), including the WhatsApp / Telegram / Facebook link-preview crawler that fetches the page to build the card. The dashboard's "קליקים על הקישור" therefore counts crawler hits as human taps.

5. iOS vs Android

	Android	iOS
Custom schemes registered	<code>footy</code> , <code>teamder</code> , <code>com.studiogameslime.soccera</code> <code>pp</code> (<code>app.json</code> <code>expo.scheme</code> ; <code>android/app/src/main/AndroidManifest.xml:51-52</code>)	same three (<code>app.json</code> <code>expo.scheme</code>)
App Links / Universal Links declared	<code>autoVerify=true</code> , <code>host</code> <code>teamderfc.web.app</code> , <code>pathPrefix</code> <code>/session</code> , <code>/team</code> , <code>/app</code> only (<code>app.json</code> <code>expo.android.intentFilters</code> ; <code>android/app/src/main/AndroidManifest.xml:55-59</code>)	<code>associatedDomains</code> : <code>["applinks:teamderfc.web.app"]</code> (<code>app.json</code> <code>expo.ios.associatedDomains</code>), entitlement present locally at <code>ios/Teamder/Teamder.entitlements</code>
Hosting side	<code>public/.well-known/assetlinks.json</code> , <code>package</code> <code>com.studiogameslime.soccera</code> <code>pp</code> , two SHA-256 fingerprints (<code>D2:73:E1:...</code> , <code>E8:72:FE:...</code>)	<code>public/.well-known/apple-app-site-association</code> , <code>appId</code> <code>M7R8M498Z9.com.studiogameslime.soccerapp</code> , <code>"paths"</code> : <code>["/session/*", "/team/*", "/app"]</code> ; served as <code>application/json</code> via <code>firebase.json:41-43</code>
Status per memory	Shipped and working since 1.0.30	<code>project_app_links_setup</code> records Universal Links as never having worked on a shipped build — the <code>associatedDomains</code> key was stripped for 1.0.30 because the provisioning profile lacked the Associated Domains capability, and a real iOS user confirmed 2026-06-30 that the link opens Safari. The key was re-added to <code>app.json</code> the same day; the entitlement is in the local prebuild. Whether the currently-served binary carries it cannot be read from this repo.
Deferred attribution	Play Install Referrer (native module <code>react-native-play-install-referrer</code> , <code>installReferrerService.ts:42</code>)	clipboard bridge (<code>clipboardInviteService.ts</code>) + a system paste prompt
Landing CTA behaviour	scheme attempt → 1.5 s → Play (<code>invite.html:231</code>)	clipboard write → App Store immediately, never attempts the scheme (<code>invite.html:231</code>)

	Android	iOS
Store buttons shown on <code>invite.html</code>	both	both — an iPhone visitor is shown <code>Google Play</code> הורד ב as the <i>first</i> button (<code>invite.html:187</code>), then <code>App Store</code> הורד ב (<code>invite.html:188</code>). No platform hiding anywhere in this file.
Store buttons on <code>/c/</code>	Play, with referrer	rewritten to App Store (<code>c/index.html:1038-1041</code> , <code>1075</code>)
Store buttons on <code>/get</code>	irrelevant button hidden, auto-redirect	irrelevant button hidden, auto-redirect (<code>get.html:59-65</code>)

The gap that matters most on both platforms: `/i/*` — the path *every* modern share produces — is **absent from** `assetlinks.json` 's effective claim set and from the AASA paths array, and absent from the Android `intentFilters`. `/go` and `/c/*` are likewise absent. So a short invite link **never** opens the installed app directly on either platform; it always renders the web landing page first, and the person must then tap a CTA that fires a custom scheme. Only the long `/session/*`, `/team/*` and `/app` forms — which the app almost never generates any more — are real App Links.

6. Opening an invite link on DESKTOP

`/i/<code>`, `/session/<id>`, `/team/<id>`, `/app`, `/go` → `invite.html`. No platform branch fires: `isIOS` is `false` (`invite.html:230`), so `primary()` takes the Android branch. Clicking any of the three CTAs assigns `location.href = 'footy://...'` — which a desktop browser answers with an "external protocol" dialog or a silent failure — and 1500 ms later the page redirects the **desktop** browser to the Google Play web listing (`invite.html:231`). A Mac user is sent to Google Play. There is no QR code anywhere in the repo (`grep` for `qr / QRCode` across `src`, `public`, `package.json` returns nothing), no "send yourself a link", no email capture. The two store buttons at the page bottom (`invite.html:187-188`) are the only usable desktop affordance, and they are below the fold on the CH-4 section.

The desktop page is otherwise fully rendered: the `@media (min-width:900px)` block (`invite.html:132-138`) widens the layout to a 900 px two-column hero, so it *looks* designed for desktop while its primary action is not.

`/c/<groupId>` → showcase renders normally (no deep-link attempt, no redirect), download buttons point at the Play web listing with `&referrer=invite_team_<id>` (`c/index.html:1070,1075`). This is the only route that reads coherently on a desktop.

`/get` → both buttons, no redirect (`get.html:66`). Coherent.

`/` → the beta page, with a `wa.me` link (opens WhatsApp Web) and a `mailto:`.

7. The landing-page copy, verbatim (`public/invite.html`)

Static defaults, shown before `/invite-preview` resolves and permanently when it fails: - `<title>` `Teamder` · פותחים משחק, החברה נרשמים ו- `Teamder` מטפלת (`:6`) - `meta description` מארגנים כדורגל בלי כאב ראש כל המשחק השבועי (`:7`) - `og:description` בכל השאר - הרשמות, רשימת המתנה, חלוקת כוחות וסטטיסטיקות. (`:10`) - `og:image` `https://teamderfc.web.app/logo.png` (`:11`) — a **512×512 square** logo, while `twitter:card` is declared `summary_large_image` (`:13`). No 1200×630 share image exists in `public/`. - header button `הורד` (`:154`) - badge `Teamder` ל- ברוכים הבאים (`:159`), `h1` `כדורגל`

פותרים משחק, החברה נרשמים ו-Teamder מטפלת בכל השאר. lead (:160) קבוע. בלי כאב הראש של הארגון. (:161) dead- (:164) חינם · Android ו-iPhone · עברית מלאה trust line (:163) הורד את CTA Teamder - (:161) link note CH2 h2 - (:165) הקישור כבר לא זמין, אבל אפשר למצוא משחקים נוספים ב-Teamder. 40 הודעות

במקום לרדוף אחרי תשובות בקבוצה, כולם רואים מי מגיע ומה מצב המשחק. sub (:172) למשחק אחד מסודר

חמישי mock order card - (:175) מי מביא כדור? / חסר לנו אחד / מי מגיע? animated chat bubbles (:173)

פחות CH3 h2 - (:178-179) הקבוצות מוכנות · אתם רק מגיעים לשחק · 10/10 שחקנים · המשחק מלא · כדורגל

store - (:185) כל מה שצריך כדי לארגן את המשחק הבא במקום אחד. sub (:184) הודעות.

תנאי פרטיות · footer © Teamder (:188) App Store הורד ב- / (:187) Google Play הורד ב- buttons

הורד את Teamder sticky bar - terms.html / , which does not exist. links to תנאי שימוש — (:191) שימוש

(:193)

dated **20.07.26** and both use the old tab labels `מסחקים / מועדונים`, while the current app vocabulary is `מחזור` (`he.ts:287`, `1577-1581`, `3012`). `public/app-preview.png` (`1080×2400`, `517 KB`) is served but referenced by nothing.

Terminology also splits across the web surface: `invite.html` says `מועדון` (`:235`), but `c/index.html` still says `קהילה` throughout (`:7`, `:13`, `:791`, `:804`, `:834`, `:843`, `:919`), and the `serveCommunityPage` OG fallback says `Teamder`-`מועדון` (`functions/src/index.ts:10537`) while `getInvitePreview` 's sibling copy is unlabelled. `public/index.html` says `קהילות סגורות` (`:463`).

8. What the person meets AFTER install, before the thing they were invited to

Worth stating here because it belongs to the acquisition cost: the stashed invite is consumed by `RootNavigator` only when `currentUser && profileComplete && hasCompletedOnboarding` are all true (`src/navigation/RootNavigator.tsx:65`). So a person who tapped "join this match" walks the entire first run — onboarding carousel, Google/Apple sign-in, profile completion, the first-run wizard — before the match they were invited to appears (`RootNavigator.tsx:146-150` → `navigationRef.ts:109-140`). A generic `type: 'app'` invite navigates **nowhere at all**: the stash is cleared and the person is left on home (`RootNavigator.tsx:77-80`), and on a warm link the same (`App.tsx:492-495`). A dead target shows a **hardcoded** Hebrew literal, not an `he.ts` key: `'הקישור לא תקין או שהפריט כבר לא קיים'` (`RootNavigator.tsx:124`).

9. Summary table

Path	Taps to install	Attribution survives?	Known breakage
A. <code>/i/<code>short link, Android</code>	4	✅ <code>invite_<type>_<id>_by_<uid></code> via Play referrer (<code>invite.html:229</code>)	~1.5 s scheme-error interval; not an App Link, so an installed app is never opened directly; crawler inflates click counts
A'. <code>/i/<code>short link, iOS</code>	4-5 + system paste prompt	⚠️ only if the person taps the CTA (clipboard, <code>invite.html:231</code>) and approves "Allow Paste?"; lost if they use the bottom App Store button (<code>:188</code>) instead	Universal Links unverified on shipped binaries; clipboard failure is silent; iPhone users are shown a Google Play button first
B. <code>/session/<id>> /
/team/<id></code> long link	4-5	same as A / A'	<code>/session/**</code> is a static rewrite → no game-specific WhatsApp preview (generic OG); only reachable when the short-code write fails
C. <code>/get</code>	3 (cheapest)	❌ none, either platform — no referrer (<code>get.html:52,61</code>), no clipboard (<code>:64</code>)	none functionally; it is simply an unattributed funnel
D. <code>/c/<groupId></code>	3-4	⚠️ Android: club only, never an inviter (<code>c/index.html:1069</code>). iOS: ❌ nothing (<code>:1075</code>)	copy still says <code>קהילה</code> throughout; auto deep-link attempt fires before the person has read anything

Path	Taps to install	Attribution survives?	Known breakage
E. <code>/app</code> or <code>/go</code> (ad / campaign)	4-5	✅ UTM (<code>utm_source</code> / <code>utm_campaign</code> / <code>g</code> / <code>l</code>) on Android (<code>invite.html:229</code> → <code>installReferrerService.ts:94-111</code>); iOS via clipboard	<code>/go</code> is not an App Link (absent from AASA + <code>intentFilters</code>); static rewrite → no SSR OG
F. Organic store search	3	❌ none by definition	in-repo listing draft (<code>app-store-metadata.md</code>) is stale: v1.0 release notes, dead <code>teamder.app</code> URLs
G. Root <code>https://teamderfc.web.app/</code>	∞ — no install path exists	n/a	a live-in-both-stores product still asks visitors to WhatsApp their email to join a closed beta (<code>index.html:556-575</code>); the community 404 state links <i>here</i> (<code>c/index.html:793</code>)
H. <code>/downloads/teamder-0.2.5.aab</code>	not installable	n/a	a phone cannot install an <code>.aab</code> ; stale 0.2.5 artefact publicly served
I. Club join code (post-install)	n/a	❌ never credits anyone	<code>inviteSheetBody</code> promises <code>הצטרף</code> <code>ישיירות</code> , but a non-member lands on a request-to-join preview (<code>navigationRef.ts:134-138</code>)

Open questions from this section

Things this reader could not settle from the code, listed rather than assumed.

- Does the currently shipped iOS binary actually carry the `com.apple.developer.associated-domains` entitlement? The key is in `app.json` (`expo.ios.associatedDomains`) and in the local prebuild at `ios/Teamder/Teamder.entitlements`, and AASA is hosted correctly with the right Content-Type (`firebase.json:41-43`) — but memory `project_app_links_setup` records it being stripped for 1.0.30 because the provisioning profile lacked the capability, and a real iOS user confirmed on 2026-06-30 that links still open Safari. Needs `codesign -d --entitlements` on the shipped IPA, or a device test.
- What the Play Store and App Store listings ACTUALLY say right now. The only listing copy in-repo (`app-store-metadata.md`) is a paste-source draft: it still carries v1.0 'What's New', references the non-existent domain `teamder.app` for Privacy Policy and Marketing URL (lines 178-179), and lists support as `lior@hippocampus.me` while the in-app support_email default is `studiogameslime@gmail.com` (`remoteConfigService.ts:53`). Needs a look at both consoles.
- Has anyone verified that the Play Install Referrer round-trip still works end to end on a real 1.1.x install? The parse side is covered by code, but the native module (`react-native-play-install-referrer`) is loaded via an indirect require that fails silently (`installReferrerService.ts:37-51`) and marks itself consumed — a missing pod would look identical to an organic install.
- Whether `/i/`, `/go` and `/c/` were ever INTENDED to be App Links / Universal Links. They are absent from both the AASA paths array and the Android `intentFilters`, which means the path every modern share produces never opens an installed app directly. Cannot tell from code whether this is a deliberate choice (keep the landing page in the funnel) or an oversight.

- Is `https://soccer-app-52b6b.web.app` publicly discoverable / indexed? It serves every page identically (firebase.json:49-85) but is not in `HOSTING_DOMAINS`, so any link on it is dead for deep-linking and attribution.
 - Real-world volume split: how many installs arrive via `/i/` vs `/get` vs organic store search vs the root page. The counters exist (`inviteLinks.clicks`, `inviteClicks/{uid}`, `adLinks`, `linkClicks`, `metrics/linkClicks`) but the numbers are in production Firestore, not in the repo — and `serveInviteCode`'s counter is inflated by link-preview crawlers (`functions/src/index.ts:10712-10733`), so the ratio of crawler hits to human taps needs measuring before any of it is trusted.
 - What the first-run onboarding wizard actually shows. It is a Joryio in-app HTML message authored in an external dashboard, not in this repo; only its submit handler is here (`src/services/onboardingService.ts`). Its screen order, copy and the w5 invite step's wording could not be quoted from source.
 - Whether the Play Console listing's screenshots match the two landing-page screenshots. The web ones (`shot-home.jpg` / `shot-match.jpg`) are dated 20.07.26 and still use the pre-rename 'משחקים'/'מועדונים' tab labels; whether the store assets are equally stale is not determinable here.
 - Does the WhatsApp in-app browser on iOS reliably permit `navigator.clipboard.writeText` inside the CTA tap? The whole iOS attribution chain depends on it and every failure is swallowed (`invite.html:231`). Needs a device test from inside WhatsApp, Telegram and Instagram's browsers specifically.
 - Is the un-wired `handleInvitePlayers` (`MatchDetailsScreen.tsx:1462`) intended to come back? It carries the only `game.visibility !== 'public'` guard in the share path; the live `handleShare` has none, so a club-only game is currently shareable to anyone. Cannot tell from code whether that is a regression or a decision.
-